



Socket Statistics — X-quang hệ thống socket

Network Thực Chiến · Series: Debug Mạng từ A–Z · Tập 03

Nội dung

1. **ss là gì?** — Tại sao thay thế `netstat`
2. **ss vs netstat** — So sánh chi tiết
3. **Flags & Cú pháp** — Bộ tham số cốt lõi
4. **Cheatsheet** — 8 nhóm lệnh thực chiến
5. **Đọc TCP States** — Ý nghĩa từng trạng thái
6. **CLOSE_WAIT & TIME_WAIT** — Phát hiện bug & tune kernel
7. **Kịch bản thực chiến** — 3 tình huống hay gặp nhất

Phần 1

ss là gì?

Vấn đề với netstat

`netstat` là công cụ kinh điển để xem socket — nhưng đã **deprecated** trên hầu hết distro Linux mới:

```
# Ubuntu/Debian mới: netstat không còn cài mặc định
netstat -tlnp
# → bash: netstat: command not found

# Lý do netstat chậm:
# Đọc từng file trong /proc/net/* → parse text → hiển thị
# Với hàng nghìn connection: rất chậm
```

ss giải quyết các vấn đề này:

- Giao tiếp trực tiếp qua **netlink socket** với kernel → nhanh hơn nhiều
- Hiển thị **TCP internals**: congestion window, RTT, retransmit counter
- Filter expression mạnh mẽ: theo state, IP, port, subnet

ss = **Socket Statistics** — thay thế chính thức của `netstat`.

ss vs netstat — So sánh chi tiết

Tiêu chí	<code>netstat</code>	<code>ss</code>
Nguồn dữ liệu	Đọc <code>/proc/net/*</code> (text parsing)	Netlink socket trực tiếp
Tốc độ	Chậm — nghẽn khi nhiều connection	Nhanh — không qua text layer
TCP internals	Không có	✓ cwnd, rtt, retrans, buffer
Filter	Hạn chế (grep)	Mạnh: expression theo state/IP/port
Trạng thái	Deprecated trên nhiều distro	Chuẩn hiện tại
Cú pháp	<code>netstat -tlnp</code>	<code>ss -tlnp</code> (tương tự — dễ chuyển đổi)

💡 *Cú pháp hầu như giống nhau. Chuyển từ `netstat` sang `ss`: chỉ cần đổi tên lệnh.*

Phần 2

Flags & Cheatsheet

Flags cốt lõi

Flag	Ý nghĩa	Ví dụ
-t	TCP connections	<code>ss -t</code>
-u	UDP connections	<code>ss -u</code>
-l	Listening sockets only	<code>ss -l</code>
-a	All (listening + connected)	<code>ss -ta</code>
-n	No DNS resolve (số thay tên)	<code>ss -tn</code>
-p	Show process (PID + tên)	<code>ss -tp</code>
-e	Extended info	<code>ss -te</code>
-i	TCP internals	<code>ss -ti</code>
-x	Unix domain sockets	<code>ss -x</code>

Lệnh vàng — nhớ 1 lệnh này là đủ:

```
ss -tlnp  # TCP + Listening + No DNS + Process
```

Cheatsheet — Nhóm 1: TCP connections

```
ss -t      # ESTABLISHED TCP connections
ss -ta     # Tất cả trạng thái TCP
ss -tan    # Không resolve DNS (nhanh hơn, dễ đọc hơn)
```

Output mẫu:

```
State      Recv-Q  Send-Q  Local Address:Port  Peer Address:Port
ESTAB      0       0       192.168.1.10:52341  142.250.196.46:443
ESTAB      0       0       192.168.1.10:45123  10.0.0.5:5432
```

Cột	Ý nghĩa
State	Trạng thái TCP hiện tại
Recv-Q	Bytes đã nhận nhưng app chưa đọc
Send-Q	Bytes đã gửi nhưng chưa được ACK
Local / Peer	Địa chỉ:port hai đầu kết nối

Cheatsheet — Nhóm 2: Port đang listen

```
ss -tlnp
```

State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port	Process
LISTEN	0	128	0.0.0.0:22	0.0.0.0:*	users: ("sshd",pid=1234,fd=3))
LISTEN	0	511	0.0.0.0:80	0.0.0.0:*	users: ("nginx",pid=5678,fd=6))
LISTEN	0	128	127.0.0.1:3306	0.0.0.0:*	users: ("mysqld",pid=910,fd=21))

Đọc output:

- `0.0.0.0:22` → SSH bind tất cả interface ✓
- `127.0.0.1:3306` → MySQL chỉ bind localhost ⚠ (không reach từ ngoài)
- `Recv-Q = 0` → Không có kết nối đang chờ xử lý (bình thường)

Cheatsheet — Nhóm 3: Tìm process theo port

```
ss -tlnp sport = :8080      # Process đang listen port 8080
ss -tlnp | grep 8080        # Tương tự, dùng grep

ss -tnp dport = :5432       # Ai đang kết nối đến PostgreSQL
ss -tnp | grep :3306        # Ai đang kết nối đến MySQL
```

Tìm process đang chiếm port — rất hay dùng khi deploy:

```
ss -tlnp sport = :80
# Nếu trống → nginx/apache chưa chạy
# Nếu có output → xem cột Process để biết PID
```

Cheatsheet — Nhóm 4: Filter theo state TCP

```
ss -t state established      # Chỉ ESTABLISHED
ss -t state time-wait        # TIME_WAIT (nhiều = high load)
ss -t state close-wait       # CLOSE_WAIT = app bug
ss -t state syn-recv         # SYN_RECV (nhiều = SYN flood?)
ss -t state listening        # Tương đương -l

# Đếm nhanh:
ss -tan state time-wait | wc -l
ss -tan state close-wait | wc -l
```

Cheatsheet — Nhóm 5: Filter theo địa chỉ / port

```
ss -t dst 10.0.0.1           # Kết nối đến IP cụ thể
ss -t dport = :443           # Kết nối đến HTTPS
ss -t sport = :3306           # Kết nối từ port MySQL
ss -t src 192.168.1.0/24      # Kết nối từ subnet
ss -t dst 10.0.0.0/8          # Kết nối đến toàn bộ subnet

# Kết hợp nhiều điều kiện:
ss -t dst 10.0.0.1 dport = :5432
```

Cheatsheet — Nhóm 6 & 7 & 8

TCP internals (advanced)

```
ss -tei      # -e: extended, -i: TCP internals  
# Hiện thị: cwnd (congestion window), rtt, retrans, send/recv buffer
```

Output mẫu:

```
ESTAB 0 0 192.168.1.10:52341 142.250.x.x:443  
cubic wscale:7,7 rto:220 rtt:20.5/2.1 cwnd:10 bytes_sent:4096 retrans:0/1
```

UDP

```
ss -uln      # UDP listening ports  
ss -uan      # Tất cả UDP
```

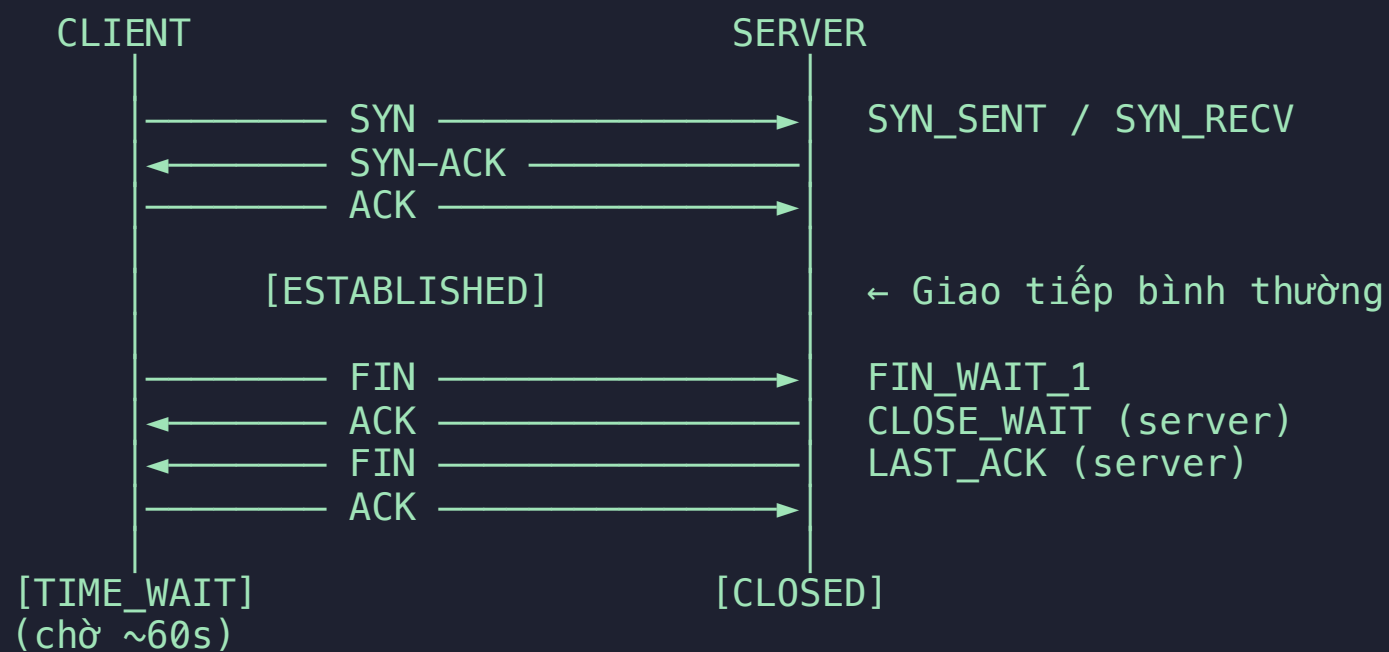
Unix domain sockets (IPC)

```
ss -xl       # Unix sockets đang listen  
ss -xp       # Unix sockets với process info
```

Phần 3

Đọc TCP States

TCP Lifecycle — Sơ đồ



Ý nghĩa TCP States

State	Ý nghĩa	Dấu hiệu bất thường
ESTABLISHED	Kết nối đang hoạt động	Quá nhiều = đang có load lớn
LISTEN	Port mở, sẵn nhận kết nối	Bình thường
SYN_RECV	Đang trong 3-way handshake	Hàng nghìn = SYN flood attack
TIME_WAIT	Đợi đảm bảo FIN-ACK tới nơi (~60s)	Hàng nghìn = cần tune kernel
CLOSE_WAIT	Server nhận FIN nhưng app chưa gọi <code>close()</code>	File descriptor leak — app bug
FIN_WAIT_1/2	Client đã gửi FIN, đợi server	Tăng liên tục = server chậm phản hồi
LAST_ACK	Server đã gửi FIN, đợi ACK cuối	Thoáng qua là bình thường

CLOSE_WAIT — Dấu hiệu app bug



Phát hiện CLOSE_WAIT leak:

```
# Đếm số CLOSE_WAIT hiện tại:
ss -tan state close-wait | wc -l

# Nếu số này TĂNG LIÊN TỤC khi chạy nhiều lần → app có bug:
watch -n 1 'ss -tan state close-wait | wc -l'
```

Nguyên nhân thường gặp:

- Connection pool không return connection về pool
- Exception handling bỏ qua bước `close()` / `disconnect()`
- HTTP client không gọi `response.close()`

TIME_WAIT — Cần tune khi high load

```
# Đếm TIME_WAIT hiện tại:  
ss -tan state time-wait | wc -l
```

TIME_WAIT > 1000 → Cần nhắc tune kernel:

```
# Xem setting hiện tại:  
sysctl net.ipv4.tcp_tw_reuse  
sysctl net.ipv4.ip_local_port_range  
  
# Tune (thêm vào /etc/sysctl.conf):  
net.ipv4.tcp_tw_reuse = 1           # Tái sử dụng TIME_WAIT socket  
net.ipv4.ip_local_port_range = 1024 65535 # Mở rộng ephemeral ports  
  
# Apply ngay:  
sysctl -p
```

⚠ TIME_WAIT là bình thường — nó đảm bảo gói tin cũ không bị nhầm với kết nối mới.
Chỉ tune khi thực sự bị thiếu port (`EADDRINUSE` errors).

Phần 4

Kịch bản thực chiến

Scenario A: "Port 3000 không listen, app có chạy không?"

Kiểm tra port cụ thể:

```
ss -tlnp | grep 3000
```

Không có output → app chưa start, hoặc bind sai interface / sai port

Kiểm tra app có đang chạy bằng tên process:

```
ss -tlnp | grep -E 'node|python|java|ruby'
```

Xem app đang listen port gì:

```
ss -tlnp | grep $(pgrep -x node)
```

Checklist debug khi port không listen:

1. `ps aux | grep app` → App có đang chạy không?
2. `ss -tlnp` → App có bind đúng port không?
3. Xem log app → Có error khi start không?
4. `ss -tlnp | grep '127.0.0.1'` → App bind localhost thay vì `0.0.0.0`?

Scenario B: "MySQL chạy nhưng không connect được từ ngoài"

```
ss -tlnp | grep 3306
```

State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port	Process
LISTEN	0	151	127.0.0.1:3306	0.0.0.0:*	mysqld

Vấn đề: MySQL đang bind `127.0.0.1` — chỉ accept kết nối từ localhost!

Giải pháp:

```
# /etc/mysql/mysql.conf.d/mysqld.cnf
[mysqld]
bind-address = 0.0.0.0      # Hoặc IP cụ thể của server
```

```
# Sau khi restart, kiểm tra lại:
ss -tlnp | grep 3306
# → LISTEN 0 151 0.0.0.0:3306 ← OK
```

💡 Pattern tương tự cho Redis (`127.0.0.1:6379`), PostgreSQL (`127.0.0.1:5432`), etc.

Scenario C: "App báo 'too many open files'"

```
# Bước 1: Đếm tổng socket đang mở
ss -tan | wc -l

# Bước 2: Tìm leak
ss -tan state close-wait | wc -l    # CLOSE_WAIT không đóng
ss -tan state time-wait | wc -l    # TIME_WAIT tích tụ

# Bước 3: Kiểm tra file descriptor limit
ulimit -n                          # Limit của process hiện tại
cat /proc/sys/fs/file-max          # Limit toàn hệ thống

# Bước 4: Xem process nào đang chiếm nhiều FD nhất
ss -tanp | grep <process-name> | wc -l
```

Giải pháp theo nguyên nhân:

Nguyên nhân	Giải pháp
CLOSE_WAIT nhiều	Fix bug trong app (không đóng connection)
TIME_WAIT nhiều	Tune <code>tcp_tw_reuse</code> , mở rộng port range
Limit quá thấp	Tăng <code>ulimit -n</code> , cấu hình <code>/etc/security/limits.conf</code>

Key Takeaways

Tình huống	Lệnh	Kết luận
Port có đang listen không?	<code>ss -tlnp grep PORT</code>	Không có = app chưa start / bind sai
App bind đúng interface chưa?	<code>ss -tlnp grep PROCESS</code>	<code>127.0.0.1</code> = chỉ local
Có CLOSE_WAIT leak không?	<code>ss -tan state close-wait wc -l</code>	Tăng liên tục = bug app
TIME_WAIT có quá nhiều không?	<code>ss -tan state time-wait wc -l</code>	>1000 = tune kernel
Ai đang kết nối đến DB?	<code>ss -tnp dport = :5432</code>	Thấy PID + process name

Bộ lệnh cốt lõi:

```
ss -tlnp          # Lệnh vàng – port nào đang listen
ss -tan state close-wait  # Phát hiện connection leak
ss -tan state time-wait   # Kiểm tra TIME_WAIT tích tụ
ss -te            # TCP internals (cwnd, rtt, retrans)
```

Cảm ơn đã theo dõi! 🙏

Network Thực Chiến

"`ss -tlnp` là lệnh đầu tiên khi debug 'tại sao không kết nối được'."